



BTS CIEL — Informatique et Réseaux

Épreuve E6

RAPPORT DE MINI-PROJET

Développement d'une solution logicielle

Session 2027

ÉTUDIANT

Nom : Nom Prénom

Parcours : BTS CIEL — IR

ENTREPRISE

Entreprise : ABCDE

Tuteur : Prof. XYZ

Année universitaire 2026–2027

Table des matières

1. Introduction	3
2. Section 1	4
2.1 Arborescence du projet `auth-backend-app`	4
2.2 Sous-section 1.2	6
3. Section 2	8
3.1 Diagrammes UML	8
3.2 Exemple : Code Python (Script de supervision)	8
3.3 Diagrammes	11
4. Section 3	12
4.1 Sous-Section 3.1	12
5. Section 4	13
6. Conclusion	14
7. Annexes.....	15
7.1 Code source.....	15
7.2 Captures d'écran	15
7.3 Glossaire	17
7.4 Acronymes	17

1. Introduction

Important - Document d'exemple Ce document est fourni uniquement comme exemple de structure et de présentation générale d'un rapport de projet. Il n'est **pas attendu** que la mise en forme soit reproduite à l'identique.

Vous êtes libres d'adapter l'organisation, la présentation et le style de vos rapports, **dans le respect des consignes du projet.**

Les contenus textuels de ce document, ainsi que les illustrations, schémas, extraits de code et autres éléments présentés, peuvent être fictifs ou constituer des éléments de remplissage (placeholder).

Ils ne doivent en aucun cas être repris, reproduits ou intégrés dans les rapports remis.

Le document est structuré comme suit :

- La [section 1](#) décrit l'analyse des besoins et la conception de l'architecture.
- La [section 2](#) détaille l'implémentation technique.
- La [section 3](#) présente les résultats et les tests effectués.
- La [section 4](#) présente ...

Le protocole [TCP](#) est utilisé pour les communications fiables.

Une attaque [DDOS](#) peut saturer un serveur.

Une entreprise peut utiliser un [VPN](#) pour sécuriser les connexions distantes.

Un [firewall](#) protège le réseau local.

2. Section 1

2.1 Arborescence du projet `auth-backend-app`

Cette partie décrit l'architecture technique de l'application backend dédiée à la gestion de l'authentification. Elle détaille l'organisation des fichiers, les composants clés, et les bonnes pratiques mises en œuvre pour assurer la maintenabilité, la scalabilité et la sécurité du projet.

L'application est organisée selon une architecture modulaire, séparant clairement les responsabilités entre les différentes couches logicielles. Voici la structure principale :

```
auth-backend-app/  
├── app/  
│   ├── api/  
│   ├── services/  
│   ├── config/  
│   ├── utils/  
│   └── main.py  
├── tests/  
│   ├── unit/  
│   └── integration/  
├── docs/  
│   ├── reports/  
│   └── recipes/  
├── database/  
│   └── schema.sql  
├── scripts/  
├── .github/  
├── .gitignore  
├── .env.example  
├── docker-compose.yml  
├── Dockerfile  
├── requirements.txt  
└── README.md
```

2.1.1 Cœur applicatif (`app/`)

- **api/** : Points d'entrée HTTP (routes et contrôleurs).
- **services/** : Logique métier (règles de gestion, traitement des données).
- **config/** : Configuration (variables d'environnement, paramètres techniques).
- **utils/** : Fonctions utilitaires (sécurité, hachage, etc.).
- **main.py** : Point d'entrée de l'application (initialisation, démarrage du serveur).

2.1.2 Suite de tests (`tests/`)

- `unit/` : Tests unitaires (validation isolée des composants).
- `integration/` : Tests d'intégration (interactions entre composants, bases de données de test).

2.1.3 Documentation (`docs/`)

- `reports/` : Documents techniques (analyses de risques, spécifications).
- `recipes/` : Fiches de recette fonctionnelle (scénarios de validation, tests métier).

2.1.4 Base de données (`database/`)

- `schema.sql` : Schéma de la base (tables, index, contraintes).

2.1.5 Configuration et déploiement

- `.github/` : Workflows GitHub Actions (CI/CD).
- `docker-compose.yml` : Orchestration des services (backend, base de données).
- `Dockerfile` : Image Docker de l'application.
- `requirements.txt` : Dépendances Python.
- `.env.example` : Template des variables d'environnement.
- `.gitignore` : Fichiers exclus du versionnage.

2.1.6 Documentation utilisateur (`README.md`)

Guide pour :

- Installer le projet.
- Lancer l'application.
- Exécuter les tests.
- Contribuer au projet.

2.1.7 Diagrammes

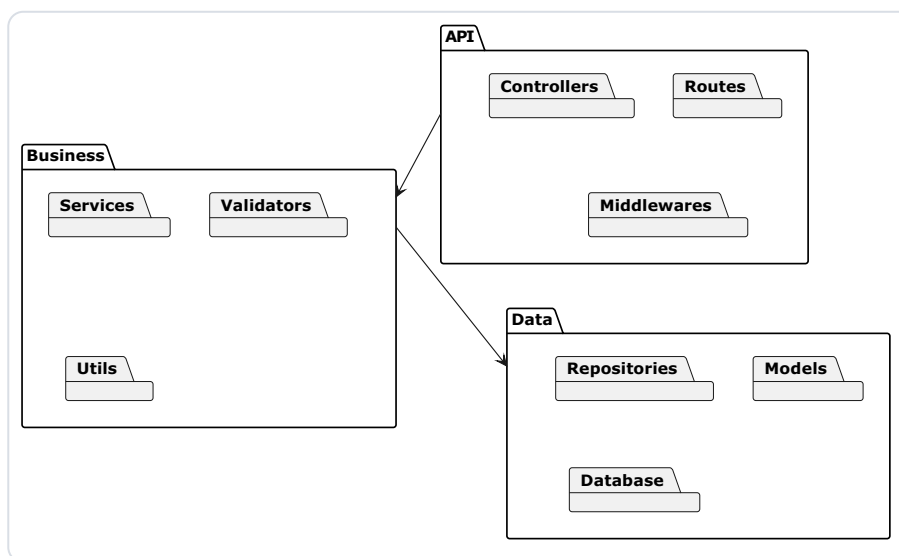


Figure 1 — Diagramme de packages

2.2 Sous-section 1.2

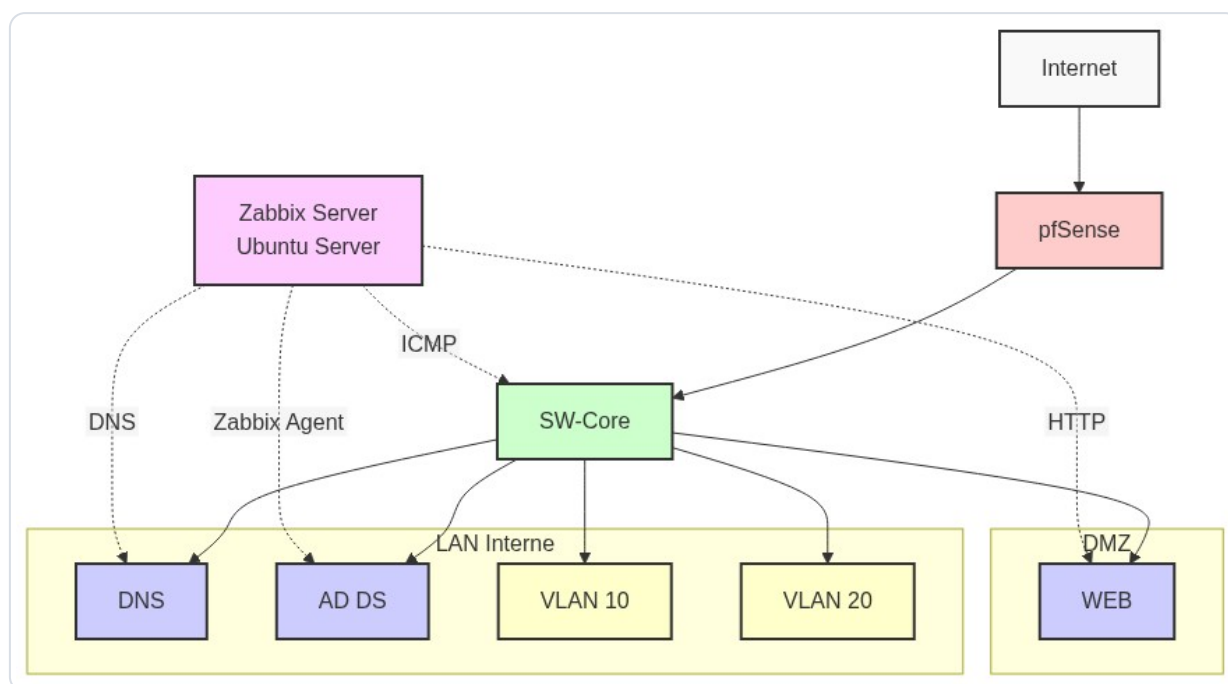


Figure 2 — Architecture de supervision réseau avec Zabbix

2.2.1 Diagrammes SysML

2.2.1.1 Diagramme de cas d'utilisation

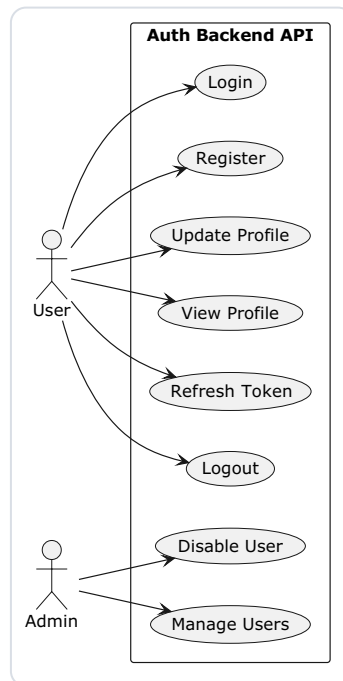


Figure 3 — Diagramme de cas d'utilisation

3. Section 2

3.1 Diagrammes UML

3.1.1 Diagramme de classes

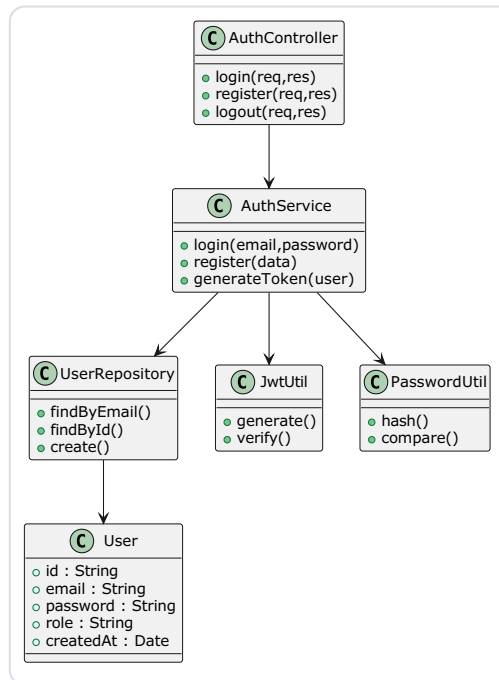


Figure 4 — Diagramme de classes

3.2 Exemple : Code Python (Script de supervision)

3.2.1 Script Python pour récupérer les métriques CPU via Zabbix API

```
import requests, json, os
from datetime import datetime

def zabbix_login():
    """Authentification auprès de l'API Zabbix."""
    payload = {
        "jsonrpc": "2.0",
        "method": "user.login",
```

```
        "params": {
            "user": ZABBIX_USER,
            "password": ZABBIX_PASSWORD
        },
        "id": 1
    }
    response = requests.post(ZABBIX_SERVER, data=json.dumps(payload), headers=HEADERS)
    return response.json()["result"]

if __name__ == "__main__":
    token = zabbix_login()
```

3.2.2 Automatisation du déploiement

Pour automatiser le déploiement de l'agent Zabbix sur tous les serveurs du réseau local, nous avons développé un script Bash. Ce script se connecte en SSH à chaque hôte, installe l'agent, et le configure pour qu'il communique avec le serveur Zabbix central.

3.2.2.1 Script Bash pour déployer l'agent Zabbix sur plusieurs serveurs

```
#!/bin/bash

# Liste des adresses IP des serveurs à configurer
SERVERS=("192.168.1.101" "192.168.1.102" "192.168.1.103")
ZABBIX_SERVER_IP="192.168.1.100"
ZABBIX_SERVER_NAME="Zabbix-Server"

# Boucle sur chaque serveur
for server in "${SERVERS[@]"; do
    echo "Configuration de l'agent Zabbix sur $server..."

    # Copie du script d'installation
    scp zabbix_agent_install.sh user@$server:/tmp/

    # Exécution du script à distance
    ssh user@$server "sudo bash /tmp/zabbix_agent_install.sh $ZABBIX_SERVER_IP $ZABBIX_SERVER_NAME"

    # Vérification du statut
    ssh user@$server "sudo systemctl status zabbix-agent"
done

echo "Déploiement terminé."
```

3.2.2.2 Explications du script

- **Ligne 4** : Liste des adresses IP des serveurs cibles.
- **Ligne 7** : Boucle `for` pour itérer sur chaque serveur.

- **Ligne 10** : Utilisation de `scp` pour copier le script d'installation.
- **Ligne 13** : Exécution du script à distance via `ssh`.
- **Ligne 16** : Vérification du statut du service Zabbix.

3.2.3 Codes Python de la BDD

Code de connexion à la BDD	Code de création de la table
<pre> 1 from flask import Flask, render_template, request 2 import mysql.connector 3 import requests 4 from IPython.display import Image 5 6 app = Flask(__name__) 7 8 # Connexion à la BDD 9 def connexion_bdd(): 10 conn = mysql.connector.connect(11 host="localhost", 12 user="root", 13 password="", 14 database="donnees_p1" 15) 16 cursor = conn.cursor() 17 return conn, cursor </pre> Figure 5 — Code de connexion à la BDD	<pre> 1 from flask import Flask, render_template, request 2 import mysql.connector 3 from IPython.display import Image, display 4 5 def creer_table(cursor): 6 cursor.execute(""" 7 CREATE TABLE IF NOT EXISTS capteur_1 (8 id INT AUTO_INCREMENT PRIMARY KEY, 9 ville VARCHAR(50), 10 date DATETIME, 11 temperature FLOAT, 12 code_meteo INT 13) 14 """) </pre> Figure 6 — Code de création de la table

Tableau 1 — Types de variables

Type	Taille en mémoire	Valeur min	Valeur max
<code>bool</code> ¹	1 octet ²	-	-
<code>byte</code>	1 octet	0	255
<code>char</code>	1 octet	-128	127
<code>int</code>	2 octets	-32768	32767
<code>unsigned int</code>	2 octets	0	65535
<code>long</code>	4 octets	-2147483648	2147483647
<code>unsigned long</code>	4 octets	0	4294967295

Type	Taille en mémoire	Valeur min	Valeur max
<code>float</code> , <code>double</code>	4 octets	$-3.4028235 \times 10^{38}$	3.4028235×10^{38}

¹ Valeurs possibles : `true`, `false`

² 1 octet = 8 bits

3.3 Diagrammes

3.3.1 Diagramme ERD (BDD)

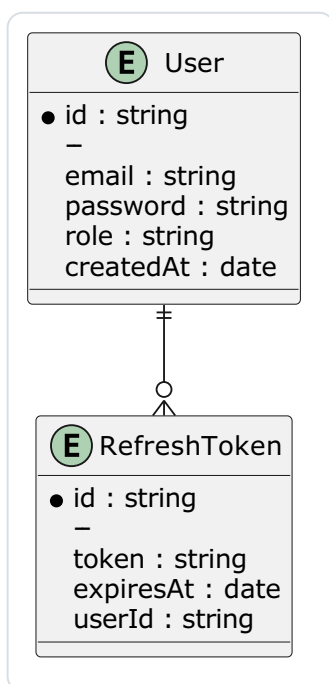


Figure 7 — Diagramme ERD (BDD)

4. Section 3

4.1 Sous-Section 3.1

Comme illustré dans l'annexe [Captures d'écran \(Interface de Zabbix\)](#), le tableau de bord Zabbix permet de visualiser en temps réel l'état des hôtes surveillés.

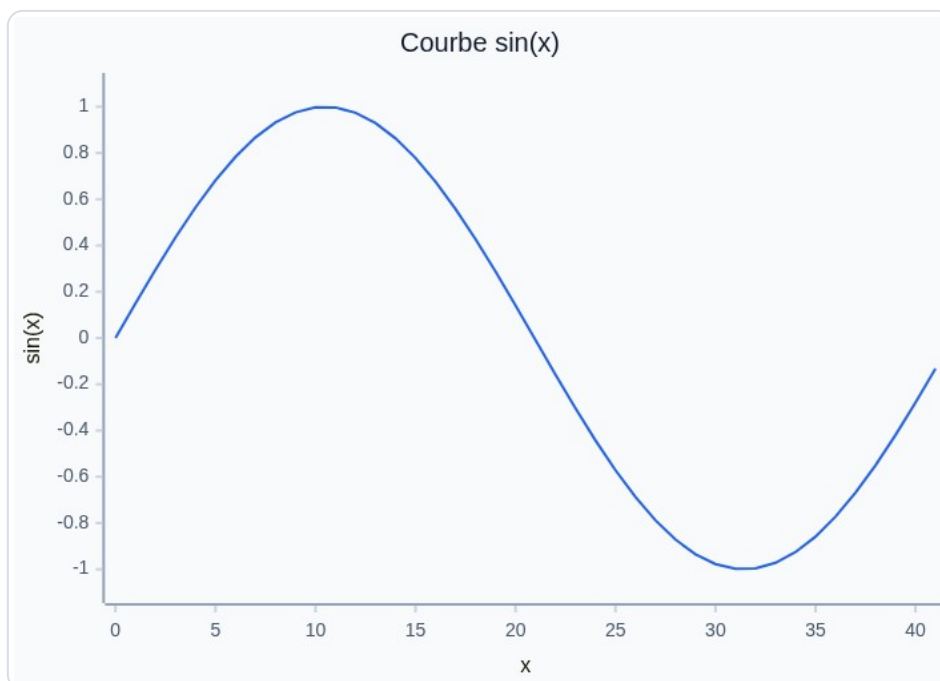


Figure 8 — Signal sinusoïdal de valeur moyenne nulle

5. Section 4

- contraintes ;
- cybersécurité ;
- difficultés rencontrées ;
- bilan personnel.

La gestion des erreurs **[1]** est essentielle.

Une autre référence : **[2]**, **[3]**, **[4]**

Le système est décrit dans **[RFC01]**.

6. Conclusion

- Résumez les résultats clés (1 paragraphe).
 - Mettez en avant les compétences acquises
(exemple : « Ce projet nous a permis de maîtriser l'outil Zabbix et la supervision réseau »).
 - Terminez par une ouverture
(exemple : « Pour aller plus loin, il serait intéressant d'intégrer... »).
-

7. Annexes

- **Glossaire** : définitions des termes techniques (ex. : Zabbix, zero-day).
 - **Références** : liens vers la documentation et les bibliothèques utilisées (ex. : FastAPI, SQLAlchemy).
-

| 7.1 Code source

7.1.1 Programmes C

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define PACKET_SIZE 64
#define TIMEOUT 5

int main(int argc, char *argv[]) {
    printf("Paquet ICMP envoyé à %s\n", argv[1]);
    return 0;
}
```

| 7.2 Captures d'écran

7.2.1 Interface de Zabbix

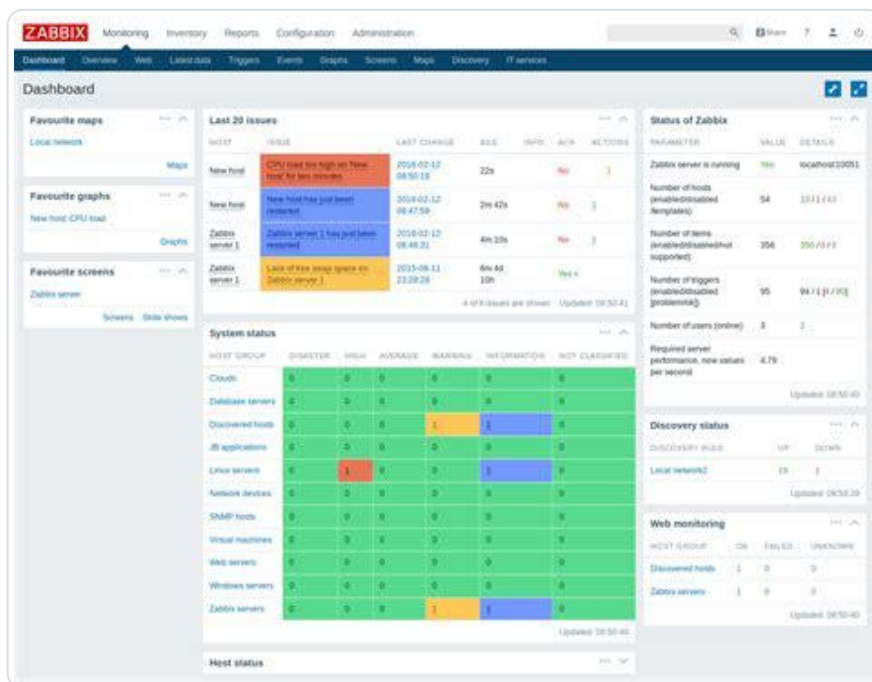


Figure 9 — Tableau de bord Zabbix — Vue d'ensemble des hôtes surveillés.

7.2.2 Codes Python

Capture	Description
<pre> 1 from flask import Flask, render_template, request 2 import mysql.connector 3 import requests 4 from IPython.display import Image 5 6 app = Flask(__name__) 7 8 # Connexion à la BDD 9 def connexion_bdd(): 10 conn = mysql.connector.connect(11 host="localhost", 12 user="root", 13 password="", 14 database="donnees_p1" 15) 16 cursor = conn.cursor() 17 return conn, cursor </pre>	Code de connexion à la BDD

Capture	Description
<pre> 1 from flask import Flask, render_template, request 2 import mysql.connector 3 from IPython.display import Image, display 4 5 def creer_table(cursor): 6 cursor.execute(""" 7 CREATE TABLE IF NOT EXISTS capteur_1 (8 id INT AUTO_INCREMENT PRIMARY KEY, 9 ville VARCHAR(50), 10 date DATETIME, 11 temperature FLOAT, 12 code_meteo INT 13) 14 """) </pre>	Code de création de la table

7.3 Glossaire

- **pare-feu**: Dispositif matériel ou logiciel filtrant les communications réseau
- **Zabbix**: Outil open source de supervision réseau et applicative

7.4 Acronymes

- **AD DS**: Active Directory Domain Services
- **API**: Application Programming Interface
- **DDoS**: Distributed Denial of Service
- **DNS**: Domain Name System
- **ICMP**: Internet Control Message Protocol
- **SQL**: Structured Query Language
- **SSH**: Secure Shell
- **TCP**: Transmission Control Protocol
- **VPN**: Virtual Private Network
- **WAN**: Wide Area Network

| Bibliographie

- [1] **A. S. Tanenbaum, H. Bos**, *Modern Operating Systems*, Pearson, 4th Edition, 2014.
- [2] **J. F. Kurose, K. W. Ross**, *Computer Networking: A Top-Down Approach*, Pearson, 8th Edition, 2021.
- [3] **W. Stallings**, *Effective Cybersecurity: A Guide to Using Best Practices and Standards*, Addison-Wesley Professional, 2017.
- [4] **L. Torvalds et al.**, *Linux Kernel Source Code Repository*, GitHub repository.
Disponible sur : <https://github.com/torvalds/linux>
- [RFC01] **FastAPI Team**, *FastAPI Documentation, Version 0.136.1*, Disponible
sur : <https://fastapi.tiangolo.com/>, Consulté en mai 2026.